

[User Guide](@id user-guide)

This guide will help you effectively use AlignedAllocs.jl in your projects.

Table of Contents

Pages = ["guide.md"]

Depth = 3

Quick Reference

Common Operations Cheat Sheet

```
# Basic allocation (cache-line aligned)
v = memalign_vec(Float64, 1000)

# Custom alignment
v = memalign_vec(Float64, 1000; align=256)

# Zero-initialized
v = memalign_clear_vec(Float64, 1000)

# Fixed-size arrays
m = memalign_fix(Float64, (10, 20))
m = memalign_clear_fix(Float64, (10, 20))

# Sequential vectors (for threading)
vecs = memalign_seq(Float64, 4, 100)

# Check alignment
a = alignment(v)

# Cache line size
cls = AlignedAllocs.CACHE_LINE_SIZE
```

When to Use What

Task	Function	Example
SIMD operations	memalign_vec	v = memalign_vec(Float64, n)
Zero-init needed	memalign_clear_vec	v = memalign_clear_vec(Int64, n)
Multi-threading	memalign_seq	vecs = memalign_seq(T, nthreads(), n)
Fixed arrays	memalign_fix	m = memalign_fix(T, dims)

Task	Function	Example
Check alignment	<code>alignment</code>	<code>a = alignment(v)</code>

Alignment Sizes

Use Case	Alignment	Example
SSE (128-bit)	16 bytes	<code>align=16</code>
AVX (256-bit)	32 bytes	<code>align=32</code>
AVX-512 (512-bit)	64 bytes	<code>align=64</code>
Cache line	64 bytes (typical)	<code>align=CACHE_LINE_SIZE</code> (default)
Page boundary	4096 bytes	<code>align=4096</code>

Basic Usage

Allocating Aligned Vectors

The simplest way to allocate aligned memory is with `memalign_vec`:

```
using AlignedAllocs

# Allocate 1000 Float64 elements aligned to cache line
v = memalign_vec(Float64, 1000)

# Use it like any Vector
v[1] = 3.14
v[end] = 2.71
sum(v)
```

The `memalign` function is an alias for `memalign_vec`:

```
v = memalign(Float64, 1000) # Same as memalign_vec
```

Custom Alignment

Specify custom alignment with the `align` keyword argument:

```
# Align to 256 bytes (useful for AVX-512)
v = memalign_vec(Float64, 1000; align=256)

# Align to 128 bytes (useful for AVX2)
v = memalign_vec(Float64, 1000; align=128)
```

```
# Align to 64 bytes (typical cache line)
v = memalign_vec(Float64, 1000; align=64)
```

!!! note "Alignment Requirements" - Alignment must be a power of 2 - Minimum alignment is 16 bytes - Default alignment is the detected cache line size (typically 64 bytes)

Zero-Initialized Allocations

Use `memalign_clear_vec` for zero-initialized memory:

```
# All elements are guaranteed to be zero
v = memalign_clear_vec(Int64, 1000)
@assert all(iszero, v)

# With custom alignment
v = memalign_clear_vec(Float64, 500; align=128)
```

The `memalign_clear` function is an alias:

```
v = memalign_clear(Float64, 1000) # Same as memalign_clear_vec
```

!!! tip "When to Zero-Initialize" - Security: Prevent information leaks - Correctness: Algorithms that rely on zero initialization - Debugging: Easier to spot uninitialized values

Skip zero-initialization when performance is critical and you'll immediately overwrite the data.

Checking Alignment

Verify the alignment of any array:

```
v = memalign_vec(Float64, 100; align=256)
a = alignment(v)
println("Alignment: $a bytes") # Should be ≥ 256
```

The `alignment` function works on any `AbstractArray`:

```
regular = Vector{Float64}(undef, 100)
println("Regular array alignment: $(alignment(regular)) bytes")

aligned = memalign_vec(Float64, 100)
println("Aligned array alignment: $(alignment(aligned)) bytes")
```

Working with Different Types

Numeric Types

AlignedAllocs works with any bits type:

```
# Integer types
v_i8  = memalign_vec(Int8, 1000)
v_i16 = memalign_vec(Int16, 1000)
v_i32 = memalign_vec(Int32, 1000)
v_i64 = memalign_vec(Int64, 1000)

# Unsigned integers
v_u8  = memalign_vec(UInt8, 1000)
v_u32 = memalign_vec(UInt32, 1000)

# Floating point
v_f32 = memalign_vec(Float32, 1000)
v_f64 = memalign_vec(Float64, 1000)

# Boolean
v_bool = memalign_vec(Bool, 1000)
```

Complex Numbers

Complex types are supported:

```
v = memalign_vec(ComplexF64, 1000)
v[1] = 1.0 + 2.0im

# Zero-initialized complex array
v = memalign_clear_vec(ComplexF32, 500)
@assert all(iszero, v)
```

Custom Struct Types

Any bits-type struct works:

```
struct Point3D
    x::Float64
    y::Float64
    z::Float64
end

# Must be a bits type
@assert isbitstype(Point3D)
```

```
# Allocate aligned array of Points
points = memalign_vec(Point3D, 1000; align=64)
points[1] = Point3D(1.0, 2.0, 3.0)
```

!!! warning "Bits Type Requirement" Only bits types (types without pointers) can be allocated with

AlignedAllocs: `julia memalign_vec(String, 100)` # ✗ Error: not a bits type

`memalign_vec(Vector{Int}, 10)` # ✗ Error: not a bits type `memalign_vec(Int64, 100)` # ☑

OK: bits type

Fixed-Size Arrays

When using `FixedSizeArrays.jl`, you can create aligned fixed-size arrays:

1D Fixed Arrays

```
using FixedSizeArrays

# Create a fixed-size vector
v = memalign_fix(Float64, 100)

# With custom alignment
v = memalign_fix(Float64, 100; align=128)

# Zero-initialized
v = memalign_clear_fix(Int32, 50)
```

Multi-Dimensional Fixed Arrays

```
# 2D array (matrix)
m = memalign_fix(Float64, (10, 20))
@assert size(m) == (10, 20)

# 3D array
arr = memalign_fix(Int32, (5, 10, 15))
@assert size(arr) == (5, 10, 15)

# Zero-initialized multi-dimensional
m = memalign_clear_fix(Float64, (8, 8))
@assert all(iszero, m)
```

Sequential Aligned Vectors

Create multiple vectors, each aligned independently:

```
# Create 5 vectors, each with 10 elements, each aligned to cache line
vecs = memalign_seq(Float64, 5, 10)

@assert length(vecs) == 5
for v in vecs
    @assert length(v) == 10
    @assert alignment(v) >= 64 # Each vector is aligned
end

# With custom alignment
vecs = memalign_seq(Int32, 4, 8; align=256)
```

!!! info "Use Case: Sequential Vectors" `memalign_seq` is useful for multi-threaded code where each thread works on a separate vector, preventing false sharing: `julia vecs = memalign_seq(Float64, Threads.nthreads(), 1000) Threads.@threads for i in 1:Threads.nthreads() # Each thread gets its own aligned vector process_data!(vecs[i]) end`

Performance Optimization

SIMD Operations

Aligned memory enables efficient SIMD vectorization:

```
using LoopVectorization

function compute_aligned!(y, x)
    @turbo for i in eachindex(y, x)
        y[i] = sin(x[i]) + cos(x[i])
    end
end

# Aligned allocations give better SIMD performance
x = memalign_vec(Float64, 10000; align=64)
y = memalign_vec(Float64, 10000; align=64)

rand!(x)
compute_aligned!(y, x)
```

Avoiding False Sharing

In multi-threaded code, align thread-local data to cache lines:

```
using Base.Threads

# Bad: False sharing likely
results = zeros(nthreads())

# Good: Each element on separate cache line
```

```

results = memalign_vec(Float64, nthreads(); align=64)

@threads for i in 1:nthreads()
    # Each thread writes to its own cache line
    results[i] = expensive_computation()
end

```

Cache-Friendly Layouts

For matrix operations, align rows to cache lines:

```

# Allocate matrix with aligned rows
function aligned_matrix{T}(rows, cols)
    stride = max(cols, 64 ÷ sizeof{T}) # At least cache line per row
    buffer = memalign_clear_vec{T}(rows * stride)
    reshape(buffer, stride, rows)
end

m = aligned_matrix{Float64}(100, 100)

```

Common Patterns

Double Buffering

```

# Allocate two buffers for double buffering
buf1 = memalign_vec(Float64, 10000)
buf2 = memalign_vec(Float64, 10000)

for iteration in 1:niters
    # Process buf1 -> buf2
    process!(buf2, buf1)
    # Swap buffers
    buf1, buf2 = buf2, buf1
end

```

Working with C Libraries

```

# Many C libraries require aligned data
v = memalign_vec(Float64, 1000; align=32)

# Pass to C function (example)
ccall{(:process_data, "mylib"), Cvoid,
      (Ptr{Float64}, Csize_t), v, length(v)}

```

Pre-allocation for Hot Loops

```
# Pre-allocate aligned buffers outside hot loop
function process_data(input)
    # Reuse these buffers
    temp1 = memalign_vec(Float64, length(input))
    temp2 = memalign_vec(Float64, length(input))

    for iter in 1:1000
        # Use pre-allocated buffers
        transform!(temp1, input)
        combine!(temp2, temp1)
    end

    return temp2
end
```

Best Practices

☒ Do

1. Use aligned allocations for SIMD-intensive code

```
data = memalign_vec(Float64, 10000)
@turbo for i in eachindex(data)
    data[i] = sqrt(data[i])
end
```

2. Align to cache lines for multi-threaded access

```
thread_data = memalign_seq(Float64, nthreads(), 1000)
```

3. Check alignment when debugging

```
@assert alignment(v) >= 64 "Data not properly aligned!"
```

4. Use zero-initialization when needed

```
accumulator = memalign_clear_vec(Int64, 1000)
```

☐ Don't

1. Don't over-align small arrays


```
# Wasteful for small arrays
small = memalign_vec(Int32, 10; align=4096) # ✗
```

2. Don't assume alignment changes data

```
# Alignment doesn't change values, only memory layout
v = memalign_vec(Float64, 100)
v[1] = 3.14 # Still need to initialize
```

3. Don't use non-bits types

```
v = memalign_vec(String, 100) # ✗ Error
```

4. Don't forget the SIMD-friendly algorithm

```
# Alignment alone doesn't help without vectorizable code
v = memalign_vec(Float64, 1000)
# This still won't vectorize well:
for i in 2:length(v)
    v[i] = v[i-1] + rand() # Data dependency
end
```

Troubleshooting

Diagnostic Flowchart

```
Performance not improving?
├─> Is alignment correct?
│   │ Yes ↓
│   └─> Check: alignment(v) >= expected
├─> Is code vectorizable?
│   │ Yes ↓
│   └─> Use @turbo or @simd
├─> Are you actually using aligned data?
│   │ Yes ↓
│   └─> Profile to find bottleneck
└─> Is the problem memory-bound?
    └─> Alignment won't help much
```

"ArgumentError: element_type must be a bitstype"

```
# ✗ Error
v = memalign_vec(String, 100)

# ✓ Solution: Use bits types only
v = memalign_vec{Int64}(100)
```

"ArgumentError: Alignment must be 2^p where p >= 4"

```
# ✗ Error
v = memalign_vec{Float64}(100; align=10)

# ✓ Solution: Use power of 2 >= 16
v = memalign_vec{Float64}(100; align=16)
```

Performance Not Improving

If you don't see performance improvements:

1. Check if your code is vectorizable

```
using LoopVectorization
# Add @turbo or @simd
```

2. Verify alignment

```
println("Alignment: $(alignment(v)) bytes")
```

3. Profile your code

```
using Profile
@profile your_function(v)
Profile.print()
```

4. Compare with regular arrays

```
using BenchmarkTools
@btime your_function($regular_array)
@btime your_function($aligned_array)
```

Common Mistakes

1. Assuming alignment changes data

```
v = memalign_vec(Float64, 100)
# Still need to initialize!
v[1] = 3.14
```

2. Over-aligning small arrays

```
# Wasteful - alignment overhead > benefit
v = memalign_vec(Int32, 4; align=4096) # ✗
```

3. Forgetting to use SIMD-friendly code

```
# Alignment alone isn't enough
@turbo for i in eachindex(v) # ☒ Add this
    v[i] = sqrt(v[i])
end
```

Migration Guide

From Regular Arrays to AlignedAllocs

Step 1: Identify Candidates

Look for:

- SIMD-intensive loops
- Multi-threaded access patterns
- C/Fortran interop
- Cache-critical code

Step 2: Simple Replacement

```
# Before
v = Vector{Float64}(undef, 1000)

# After
v = memalign_vec(Float64, 1000)

# Before (with zeros)
v = zeros(Float64, 1000)
```

```
# After
v = memalign_clear_vec(Float64, 1000)
```

Step 3: Add SIMD Annotations

```
# Before
for i in eachindex(v)
    v[i] = sqrt(abs(v[i]))
end

# After (with alignment AND SIMD)
using LoopVectorization
@turbo for i in eachindex(v)
    v[i] = sqrt(abs(v[i]))
end
```

Step 4: Measure Improvement

```
using BenchmarkTools

# Benchmark old version
old_time = @belapsed your_function($old_array)

# Benchmark new version
new_time = @belapsed your_function($aligned_array)

# Calculate speedup
speedup = old_time / new_time
println("Speedup: $(round(speedup, digits=2))x")
```

Common Patterns

Pattern 1: Pre-allocated Buffers

```
# Before
function process(data)
    temp = similar(data)
    # ... computation ...
end

# After
function process(data)
    temp = memalign_vec(eltype(data), length(data))
    # ... computation ...
end
```

Pattern 2: Thread-Local Storage

```
# Before (false sharing risk)
results = zeros(Threads.nthreads())

# After (aligned, no false sharing)
results = memalign_vec(Float64, Threads.nthreads(); align=64)
```

Pattern 3: Matrix Allocations

```
# Before
A = Matrix{Float64}(undef, m, n)

# After (aligned rows)
function aligned_matrix(m, n, T=Float64)
    stride = max(n, 64 ÷ sizeof(T))
    buffer = memalign_vec(T, m * stride)
    reshape(buffer, stride, m)'[1:m, 1:n]
end
A = aligned_matrix(m, n)
```

Performance Expectations

Realistic Speedups:

- SIMD loops: 1.2x - 3x
- Multi-threaded (avoiding false sharing): 1.1x - 2x
- Cache-critical code: 1.1x - 1.5x

When you won't see improvement:

- Memory-bound code (already saturating bandwidth)
- Non-vectorizable algorithms
- Code dominated by branches/conditionals
- Very small arrays (overhead > benefit)

Gradual Migration Strategy

1. **Start with hotspots** (identified by profiling)
2. **Migrate one function at a time**
3. **Benchmark each change**
4. **Keep only beneficial changes**

```
# Example migration
function hotspot(data)
```

```
# Step 1: Profile shows this is slow
temp = Vector{Float64}(undef, length(data))

# Step 2: Try aligned allocation
temp = memalign_vec(Float64, length(data))

# Step 3: Add SIMD
@turbo for i in eachindex(data, temp)
    temp[i] = compute(data[i])
end

# Step 4: Benchmark
return temp
end
```

Examples

"ArgumentError: element_type must be a bitstype"

```
# ✗ Error
v = memalign_vec(String, 100)

# ☑ Solution: Use bits types only
v = memalign_vec{Int64}(100)
```

"ArgumentError: Alignment must be 2^p where p >= 4"

```
# ✗ Error
v = memalign_vec{Float64}(100; align=10)

# ☑ Solution: Use power of 2 >= 16
v = memalign_vec{Float64}(100; align=16)
```

Performance Not Improving

If you don't see performance improvements:

1. Check if your code is vectorizable

```
using LoopVectorization
# Add @turbo or @simd
```

2. Verify alignment

```
println("Alignment: $(alignment(v)) bytes")
```

3. Profile your code

```
using Profile
@profile your_function(v)
Profile.print()
```

4. Compare with regular arrays

```
using BenchmarkTools
@btime your_function($regular_array)
@btime your_function($aligned_array)
```

Examples

Example 1: Matrix Multiplication

```
using LinearAlgebra

function fast_matmul(A, B)
    m, n = size(A, 1), size(B, 2)
    C = memalign_clear_vec(Float64, m * n)
    C_mat = reshape(C, m, n)

    mul!(C_mat, A, B)
    return C_mat
end

A = memalign_vec(Float64, 100 * 100) |> x -> reshape(x, 100, 100)
B = memalign_vec(Float64, 100 * 100) |> x -> reshape(x, 100, 100)
C = fast_matmul(A, B)
```

Example 2: Parallel Reduction

```
using Base.Threads

function parallel_sum(data)
    # One accumulator per thread, aligned to prevent false sharing
    accumulators = memalign_clear_vec(Float64, nthreads(); align=64)

    @threads for i in eachindex(data)
        tid = threadid()
        accumulators[tid] += data[i]
    end
    return sum(accumulators)
```

```
end

return sum(accumulators)
end

data = memalign_vec(Float64, 1_000_000)
rand!(data)
total = parallel_sum(data)
```

Example 3: Image Processing

```
function convolve2d(img, kernel)
    h, w = size(img)
    kh, kw = size(kernel)

    # Allocate aligned output
    output = memalign_clear_vec(Float64, h * w)
    out_mat = reshape(output, h, w)

    # Apply convolution
    for i in 1+kh÷2:h-kh÷2, j in 1+kw÷2:w-kw÷2
        val = 0.0
        for ki in 1:kh, kj in 1:kw
            val += img[i+ki-kh÷2-1, j+kj-kw÷2-1] * kernel[ki, kj]
        end
        out_mat[i, j] = val
    end

    return out_mat
end
```

Next Steps

- Read the [Technical Guide](@ref technical-guide) for implementation details
- See the [API Reference](@ref api-reference) for complete function documentation
- Check the examples directory for more use cases

Have questions? Open an issue on GitHub!